



Topic Modeling

--> Latent Dirichlet Allocation (LDA)

- Use of CountVectorizer

```
In [1]: # Import necessary libraries
import numpy as np
import pandas as pd

#Read the data
reviews_datasets=pd.read_csv("Reviews.csv")
reviews_datasets=reviews_datasets.head(20000)
reviews_datasets.dropna()
```

Out[1]:

	Id	ProductId	UserId	ProfileName	HelpfulnessNumer
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	
...	...	...	...	...	
19995	19996	B002C50X1M	A1XRXZI5KOMVDD	KAF1958 "amandaf0626"	
19996	19997	B002C50X1M	A7G9M0IE7LABX	Kevin	
19997	19998	B002C50X1M	A38J5PRUDESMTZF	ray	
19998	19999	B002C50X1M	A17TPOSAG43GSM	Herrick	

	Id	ProductId	UserId	ProfileName	HelpfulnessNumer
	19999	20000	B002C50X1M	A3LWC833HQIG7J	austin_Larry

19999 rows × 10 columns

```
In [2]: reviews_datasets['Text'][360]
```

```
Out[2]: "Go ahead and drink the other brands served at your neighborhood cafe if you
must but make sure you have a stash of Weaver's Peru Organic Whole Bean at ho
me to serve up to your friends and family who deserve the best coffee. And w
hen you go back to that corner cafe where people drink average coffee because
they don't know better, take along a travel mug filled with Weaver's coffee f
rom home and enjoy a real Cup of Joe. Er, Weaver's."
```

```
In [3]: from sklearn.feature_extraction.text import CountVectorizer

count_vect = CountVectorizer(max_df=0.8,min_df=2,stop_words='english')
doc_term_matrix = count_vect.fit_transform(reviews_datasets['Text'].values.ast
```

```
In [4]: doc_term_matrix
```

```
Out[4]: <Compressed Sparse Row sparse matrix of dtype 'int64'
with 594703 stored elements and shape (20000, 14546)>
```

```
In [5]: from sklearn.decomposition import LatentDirichletAllocation
LDA = LatentDirichletAllocation(n_components=5, random_state=42)
LDA.fit(doc_term_matrix)
```

Out[5]:

▼ LatentDirichletAllocation ⓘ ?

Parameters

 <code>n_components</code>	5
 <code>random_state</code>	42
 <code>doc_topic_prior</code>	None
 <code>topic_word_prior</code>	None
 <code>learning_method</code>	'batch'
 <code>learning_decay</code>	0.7
 <code>learning_offset</code>	10.0
 <code>max_iter</code>	10
 <code>batch_size</code>	128
 <code>evaluate_every</code>	-1
 <code>total_samples</code>	1000000.0
 <code>perp_tol</code>	0.1
 <code>mean_change_tol</code>	0.001
 <code>max_doc_update_iter</code>	100
 <code>n_jobs</code>	None
 <code>verbose</code>	0

Fitted attributes

Name	Type	Value
<code>bound__</code>	float64	1791
<code>components__</code>	ndarray[float64](5, 14546)	[[4.09, 0.2 , 0.21,..., 0.2 , 0.2 , 0.2], [11.55, 15.53, 0.33,..., 0.21, 0.2 , 0.2], [22.73, 13.89, 1.57,..., 0.2 , 0.2 , 0.2 , [19.89, 4.17, 0.2 , ..., 10.19, 73.2 , 25.2], [146.75, 0.2 , 3.69,..., 0.2 , 0.2 , 0.2]]
<code>doc_topic_prior__</code>	float	0.2
<code>exp_dirichlet_component__</code>	ndarray[float64](5, 14546)	[[0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.], [0.,0.,0.,...,0.,0.,0.]]
<code>n_batch_iter__</code>	int	11

```
In [6]: import random
        for i in range(10):
            random_id = random.randint(0, len(count_vect.get_feature_names_out()))
            print(count_vect.get_feature_names_out()[random_id])
```

tech
retained
22oz
schools
jar
watery
portions
trot
gets
frosting

```
In [7]: first_topic = LDA.components_[0]
```

```
In [8]: top_topic_words = first_topic.argsort()[-10:]
```

```
In [9]: for i in top_topic_words:
        print(count_vect.get_feature_names_out()[i])
```

water
great
just
drink
sugar
good
flavor
taste
like
tea

```
In [10]: for i, topic in enumerate(LDA.components_):
        print(f'Top 10 words for topic #{i}: ')
        print([count_vect.get_feature_names_out()[i] for i in topic.argsort()[-10:]])
        print('\n')
```

Top 10 words for topic #0:

```
['water', 'great', 'just', 'drink', 'sugar', 'good', 'flavor', 'taste', 'like', 'tea']
```

Top 10 words for topic #1:

```
['br', 'chips', 'love', 'flavor', 'chocolate', 'just', 'great', 'taste', 'good', 'like']
```

Top 10 words for topic #2:

```
['just', 'drink', 'orange', 'sugar', 'soda', 'water', 'like', 'juice', 'product', 'br']
```

Top 10 words for topic #3:

```
['gluten', 'eat', 'free', 'product', 'like', 'dogs', 'treats', 'dog', 'br', 'food']
```

Top 10 words for topic #4:

```
['cups', 'price', 'great', 'like', 'amazon', 'good', 'br', 'product', 'cup', 'coffee']
```

```
In [11]: topic_values = LDA.transform(doc_term_matrix)
         topic_values.shape
```

```
Out[11]: (20000, 5)
```

```
In [12]: reviews_datasets['Topic'] = topic_values.argmax(axis=1)
```

```
In [13]: reviews_datasets.head()
```

Out[13]:		Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	Help
	0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian		1
	1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa		0
	2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"		1
	3	4	B000UA0QIQ	A395BORC6FGVXV	Karl		3
	4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"		0

--> Non-Negative Matrix Factorization (NMF)

- Use of TF-IDF

```
In [14]: # Import necessary libraries
import numpy as np
import pandas as pd

#Read the data
reviews_datasets=pd.read_csv("Reviews.csv")
reviews_datasets=reviews_datasets.head(20000)
reviews_datasets.dropna()
```

Out[14]:

	Id	ProductId	UserId	ProfileName	HelpfulnessNumer
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	
...	...	...	...	...	
19995	19996	B002C50X1M	A1XRXZI5KOMVDD	KAF1958 "amandaf0626"	
19996	19997	B002C50X1M	A7G9M0IE7LABX	Kevin	
19997	19998	B002C50X1M	A38J5PRUDESMTZF	ray	
19998	19999	B002C50X1M	A17TPOSAG43GSM	Herrick	

	Id	ProductId	UserId	ProfileName	HelpfulnessNumer
19999	20000	B002C50X1M	A3LWC833HQIG7J	austin_Larry	

19999 rows × 10 columns

```
In [18]: from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vect = TfidfVectorizer(max_df=0.8,min_df=2,stop_words='english')
doc_term_matrix = tfidf_vect.fit_transform(reviews_datasets['Text'].values.astype('U'))
```

```
In [20]: from sklearn.decomposition import NMF

nmf = NMF(n_components=5, random_state=42)
nmf.fit(doc_term_matrix)
```

Out[20]:

▼ NMF ⓘ ?

Parameters

	n_components	5
	random_state	42
	init	None
	solver	'cd'
	beta_loss	'frobenius'
	tol	0.0001
	max_iter	200
	alpha_W	0.0
	alpha_H	'same'
	l1_ratio	0.0
	verbose	0
	shuffle	False

Fitted attributes

Name	Type	Value
components__	ndarray[float64](5, 14546)	[[0.05,0. ,0. ,...,0. ,0. ,0.], [0.01,0. ,0. ,...,0. ,0. ,0.], [0.02,0. ,0. ,...,0. ,0. ,0.], [0. ,0. ,0. ,...,0. ,0. ,0.], [0.01,0. ,0. ,...,0. ,0.06,0.02]]
n_components__	int	5
n_features_in__	int	14546
n_iter__	int	26
reconstruction_err__	float64	137.4

► 5 features 

nmf0
nmf1
nmf2
nmf3
nmf4

```
In [22]: import random

for i in range(10):
```

```
random_id = random.randint(0, len(tfidf_vect.get_feature_names_out()))
print(tfidf_vect.get_feature_names_out()[random_id])
```

delobas
eagerness
stiff
theirs
sorely
nabbed
column
wuss
spilling
quests

```
In [23]: first_topic = nmf.components_[0]
top_topic_words = first_topic.argsort()[-10:]
```

```
In [25]: for i in top_topic_words:
print(tfidf_vect.get_feature_names_out()[i])
```

really
chocolate
love
flavor
just
product
taste
great
good
like

```
In [26]: for i, topic in enumerate(nmf.components_):
print(f'Top 10 words for topic #{i}:')
print([tfidf_vect.get_feature_names_out()[i] for i in topic.argsort()[-10:]])
print('\n')
```

Top 10 words for topic #0:

```
['really', 'chocolate', 'love', 'flavor', 'just', 'product', 'taste', 'great',  
'good', 'like']
```

Top 10 words for topic #1:

```
['like', 'keurig', 'roast', 'flavor', 'blend', 'bold', 'strong', 'cups', 'cup',  
'coffee']
```

Top 10 words for topic #2:

```
['com', 'amazon', 'orange', 'switch', 'water', 'drink', 'soda', 'sugar', 'juic  
e', 'br']
```

Top 10 words for topic #3:

```
['bags', 'flavor', 'drink', 'iced', 'earl', 'loose', 'grey', 'teas', 'green',  
'tea']
```

Top 10 words for topic #4:

```
['old', 'love', 'cat', 'eat', 'treat', 'loves', 'dogs', 'food', 'treats', 'do  
g']
```

```
In [27]: topic_values = nmf.transform(doc_term_matrix)
reviews_datasets['Topic'] = topic_values.argmax(axis=1)
reviews_datasets.head()
```

Out[27]:

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	Help
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian		1
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa		0
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"		1
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl		3
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"		0

In []: